MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova Faculty of Computers, Informatics and Microelectronics Department of Software and Automation Engineering

Postoronca Dumitru FAF-233

Report

Laboratory work n.4

on Embedded Systems

Checked by:
A. Martiniuc, *university assistant*
DISA, FCIM, UTM

Chișinău – 2026

1 Purpose of laboratory work

Purpose of the laboratory work is to port the non-preemptive bare-metal multitasking application from Laboratory Work 3 to a preemptive FreeRTOS-based system running on the ESP32 microcontroller, with minimal changes to the task logic, while extending it with synchronization mechanisms for safe inter-task communication.

1.1 Objectives of the laboratory work

Laboratory work has the following objectives that should be achieved:

- Porting the bare-metal cooperative application to FreeRTOS preemptive multitasking with minimal modifications to the task functions themselves
- Using `vTaskDelay` and `vTaskDelayUntil` for task recurrence and offset, replacing the hardware timer ISR and tick counter
- Extending the scheduler abstraction to create and manage FreeRTOS tasks transparently, keeping task functions as plain one-shot functions identical in structure to the bare-metal version
- Implementing inter-task synchronization using a **binary semaphore** for event signaling between tasks and a **mutex** for protecting shared variables
- Demonstrating safe access to shared global resources under concurrent preemptive execution
- Measuring button press durations and classifying them as short ($< 500\,\text{ms}$) or long ($\geq 500\,\text{ms}$), preserving the same application behavior as in the bare-metal version
- Providing visual feedback through LEDs and periodic statistical reports through STDIO

1.2 Problem definition

The application must be structured in at least 3 distinct tasks:

1. **Task 1 – Button detection and press duration measurement:** Monitors the button state, detects pressed/released transitions, and measures the duration of each press in milliseconds. On each valid press, it saves the duration in a global variable (protected by mutex) and signals visually:

- Green LED on for a short press (< 500 ms)
- Red LED on for a long press (≥ 500 ms)

After updating the shared data, it gives a binary semaphore to signal Task 2.

2. **Task 2 – Press counting and statistics:** On each tick, non-blockingly checks the binary semaphore. If a new press was signaled, reads shared press data under mutex, increments counters under mutex, and performs a rapid blink of the yellow LED: 5 blinks for a short press and 10 blinks for a long press.

3. **Task 3 – Periodic reporting:** Every 10 seconds, acquires the mutex to snapshot and atomically reset all statistics counters, then transmits via STDIO: total number of presses, number of short presses (< 500 ms), number of long presses (≥ 500 ms), and average press duration.

2 Domain Analysis

2.1 Application Context and Utilized Technologies

The primary objective of this laboratory work is to implement a **preemptive multitasking system** using **FreeRTOS** on the ESP32 microcontroller. This approach extends the bare-metal cooperative scheduler from Laboratory Work 3 by introducing a real-time operating system that provides true preemptive scheduling, inter-task synchronization primitives, and drift-corrected periodic execution.

The application uses FreeRTOS **preemptive tasks** managed by the RTOS kernel scheduler. Each task runs as an independent execution context with its own stack. Periodic execution is achieved through `vTaskDelayUntil`, which provides precise, drift-corrected intervals. An initial `vTaskDelay` is used to apply the startup offset before the periodic loop begins, mirroring the offset field of the bare-metal `TaskContext` structure.

Task synchronization is achieved through two FreeRTOS primitives:

- A **binary semaphore** used by Task 1 to signal Task 2 when a new press event occurs
- A **mutex** used by Tasks 1, 2, and 3 to protect shared statistical variables from concurrent access

Communication with the user is achieved through the standard `printf()` function, which on ESP32 is natively routed to the UART interface without requiring the `fdevopen` redirection needed on AVR.

2.2 Hardware and Software Components

Component	Description & Role
ESP32 DevKit	Dual-core microcontroller board with Xtensa LX6 that runs FreeRTOS and all application tasks preemptively.
Push Button	Input peripheral connected to GPIO 15 with internal pull-up resistor. Used to detect press/release transitions for duration measurement.
Green LED	Connected to GPIO 13. Lights up to indicate a short button press (< 500 ms).
Red LED	Connected to GPIO 26. Lights up to indicate a long button press (≥ 500 ms).
Yellow LED	Connected to GPIO 32. Performs rapid blinking to acknowledge each press (5 blinks for short, 10 for long).
220 Ω **Resistors**	Current limiting resistors for each LED to keep current within safe operating limits.
Serial-to-USB Interface	Facilitates UART communication between the MCU and the terminal for periodic report output at 115200 baud.
FreeRTOS	Real-time operating system built into the ESP32 Arduino core. Provides preemptive task scheduling, binary semaphores, and mutexes.
Neovim + Arduino-nvim plugin	Development environment with `arduino-cli` integration for compilation, upload, and monitoring.
Breadboard	Prototyping base for interconnecting the button, LEDs, resistors, and the ESP32 without soldering.

2.3 System Architecture and Solution Justification

The system adopts a **preemptive FreeRTOS scheduler architecture**. Unlike the cooperative bare-metal design from Laboratory Work 3, FreeRTOS can interrupt any running task when a higher-priority task becomes ready, making the system more responsive and realistic for concurrent embedded applications.

The key architectural insight is that the **task functions themselves remain unchanged** – they are still plain, one-shot functions with no internal loops. The periodicity and offset logic is encapsulated entirely inside the scheduler service, which wraps each task function in a FreeRTOS `periodicTaskRunner`:

```
static void periodicTaskRunner(void *pvParameters) {
  TaskWrapper *ctx = (TaskWrapper *)pvParameters;
  vTaskDelay(pdMS_TO_TICKS(ctx->offsetMs));        // apply offset
   once
  TickType_t xLastWakeTime = xTaskGetTickCount();
  for (;;) {
    ctx->func();                                    // call plain
   task function
    vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(ctx->periodMs));
  }
}
```

This design minimizes changes between the bare-metal and FreeRTOS versions, satisfying the portability requirement of the laboratory work.

The data flow is structured as follows:

1. **FreeRTOS Scheduler:** Preemptively runs Task 1 every 10 ms, Task 2 every 50 ms, and Task 3 every 10 seconds based on their configured periods and priorities.

2. **Task 1:** Detects button transitions, writes shared press data under mutex, lights the appropriate LED, and gives the binary semaphore.

3. **Task 2:** Non-blockingly checks the semaphore. If taken, reads press data under mutex, updates statistics under mutex, and drives the yellow LED blink sequence.

4. **Task 3:** Acquires the mutex to atomically snapshot and reset all counters, then prints the report via `printf()`.

2.4 Relevant Case Study

A real-world application of preemptive multitasking with synchronization is found in **industrial sensor monitoring systems** with safety requirements. Unlike cooperative systems, preemptive scheduling ensures that high-priority tasks (e.g., emergency stop detection) can immediately interrupt lower-priority tasks. FreeRTOS mutexes prevent data corruption when multiple tasks share sensor readings or counters – a critical requirement in IEC 61508-compliant safety systems. The binary semaphore pattern used here (producer signals consumer without busy-waiting) is a standard technique in event-driven embedded firmware.

3 Design

3.1 Architectural sketch

Below is the architectural sketch of the components structured in the diagram that highlights the software and hardware components.

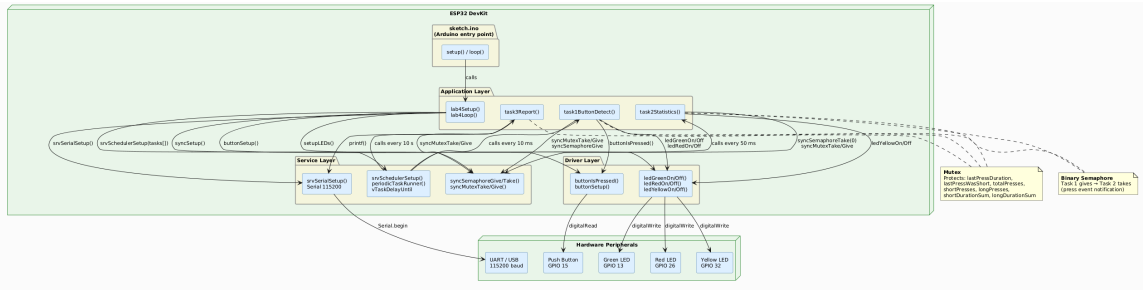

Figure 1: Architectural sketch of the project

This is the description of each of the elements of the diagram:

1. **ESP32 DevKit microcontroller** – central processing unit board that runs FreeRTOS and executes all application tasks preemptively
2. **Button (GPIO 15)** – input peripheral with internal pull-up resistor, used by Task 1 to detect press/release transitions
3. **Green LED (GPIO 13)** – output indicator for short button presses (< 500 ms)
4. **Red LED (GPIO 26)** – output indicator for long button presses (≥ 500 ms)
5. **Yellow LED (GPIO 32)** – output indicator that blinks rapidly to acknowledge each press
6. **Scheduler module** – software service that creates FreeRTOS tasks from plain task function descriptors, applying period and offset via `vTaskDelayUntil` and `vTaskDelay`
7. **Sync module** – software service that wraps a FreeRTOS binary semaphore (event signaling) and a mutex (shared variable protection) behind a simple API
8. **Serial module** – software driver that initializes UART at 115200 baud; on ESP32, `printf()` is natively routed to the serial interface

9. **LED driver module** – software driver providing abstractions for controlling each LED independently
10. **Button driver module** – software driver providing button state reading

3.2 Electrical sketch

The circuit diagram outlines the setup for this project, showing the connections between the ESP32, three LEDs (green, red, yellow), a push button, and the computer for serial communication. Each LED is connected to its respective GPIO pin through a 220 Ω current-limiting resistor. The push button is connected to GPIO 15 with the internal pull-up resistor enabled, meaning the pin reads LOW when the button is pressed.

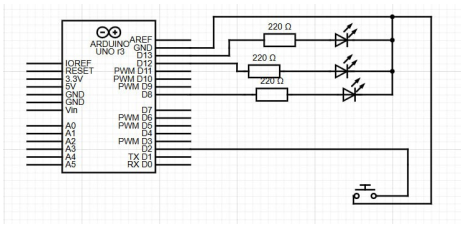

Figure 2: Circuit sketched on wokwi

Figure 2 details the connections and components that make up the system. This visual documentation is crucial for replicating the setup or diagnosing issues, as it clearly shows how each component is integrated into the overall design.

3.3 Flow chart

The Figure 3 illustrates the FreeRTOS preemptive scheduling flow. The RTOS kernel manages three independent tasks, each running on its own period. Task 1 polls the button every 10 ms and, on a press release, writes shared data under mutex and gives the binary semaphore. Task 2 runs every 50 ms and non-blockingly checks the semaphore; if taken, it reads press data and updates statistics under mutex, then drives the yellow LED. Task 3 runs every 10 seconds, acquires the mutex to snapshot and reset statistics, and prints the report.

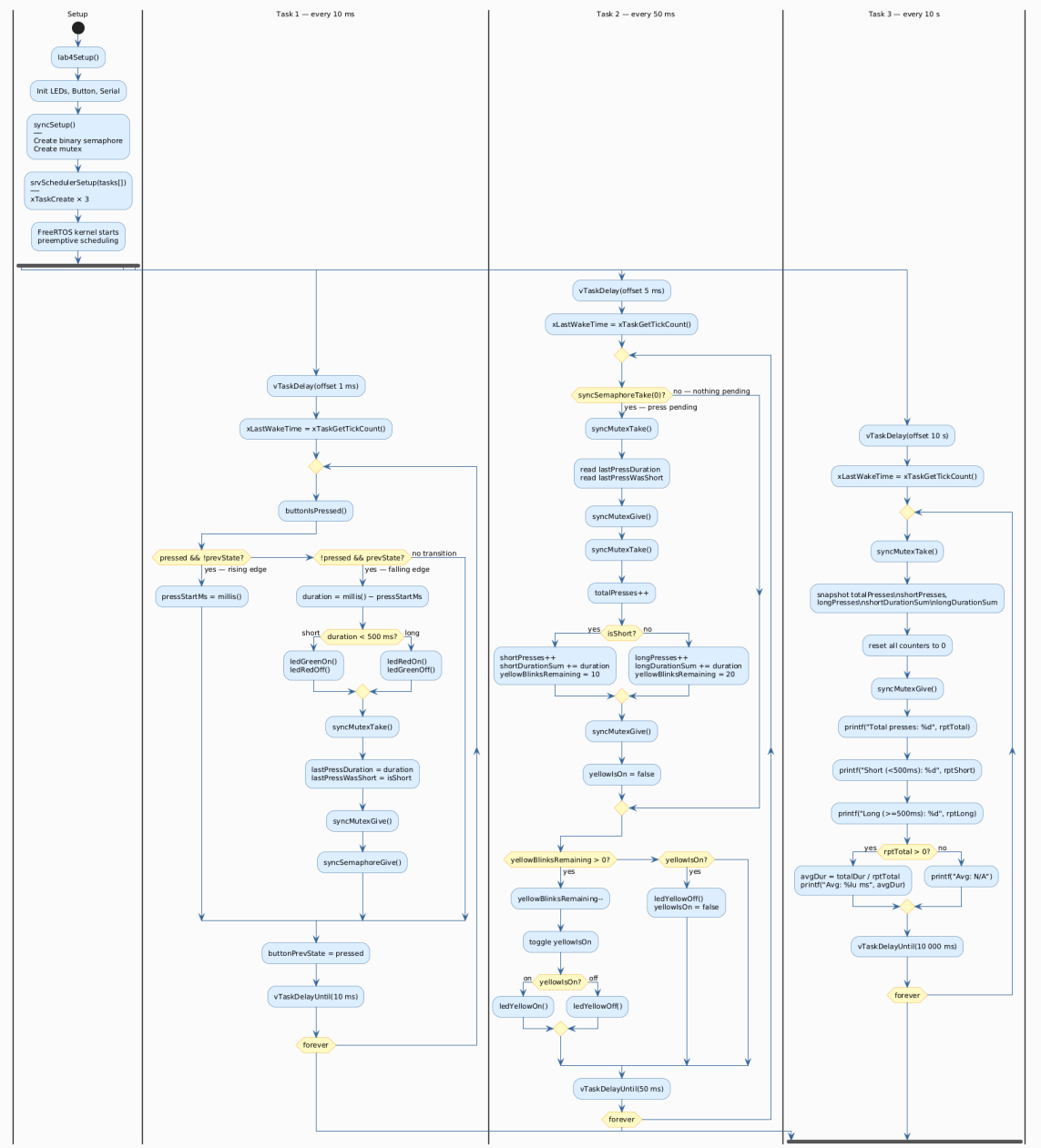

Figure 3: Flow chart of the algorithm

3.4 Code design

Code is developed following the principle of separating the interface from the implementation, by creating for each header file `.h` a respective `.cpp` file that implements it. The project is organized into three layers:

1. **Driver layer** – contains hardware-specific code for the button (`button.h/cpp`) and LEDs (`led.h/cpp`). All direct `Arduino.h` calls are encapsulated here.
2. **Service layer** – contains two reusable, hardware-independent modules: the scheduler (`scheduler.h/cpp`) which creates and manages FreeRTOS tasks, and the sync service (`sync.h/cpp`) which encapsulates the binary semaphore and mutex. The serial service (`serial.h/cpp`) initializes UART.
3. **Application layer** – contains the task logic (`lab4.h/cpp`) which defines the three plain one-shot task functions, their period/offset/priority parameters, shared global variables, and the setup routine.

The scheduler uses a `TaskDef` structure array passed to `srvSchedulerSetup()`, which creates a FreeRTOS task for each entry:

```
typedef struct {
  const char      *name;
  TaskFunc         func;       // plain void() function -- same
   shape as bare-metal
  uint32_t         periodMs;   // recurrence period
  uint32_t         offsetMs;   // initial delay before first run
  uint32_t         stackSize;
  UBaseType_t      priority;
} TaskDef;
```

Internally, the scheduler wraps each plain function in a `periodicTaskRunner` that applies the offset via `vTaskDelay` and then loops with `vTaskDelayUntil` for drift-corrected periodic execution. This design keeps the infinite loop entirely inside the scheduler abstraction, not inside the task functions.

Tasks communicate and synchronize through the sync service abstraction:

```
// Binary semaphore -- event signaling (Task 1 -> Task 2)
void syncSemaphoreGive();
bool syncSemaphoreTake(uint32_t timeoutMs);  // timeoutMs=0 is non-
   blocking

```

```
// Mutex -- shared variable protection (Tasks 1, 2, 3)
void syncMutexTake();
void syncMutexGive();
```

Task 1 calls `syncMutexTake/Give` to write `lastPressDuration` and `lastPressWasShort`, then calls `syncSemaphoreGive` to notify Task 2. Task 2 calls `syncSemaphoreTake(0)` (non-blocking, timeout = 0) to check for a pending event, then uses the mutex to read press data and update counters. Task 3 uses the mutex to atomically snapshot and reset all statistics counters before printing, ensuring no partial update from Task 2 is observed.

This design ensures that `printf()` is called from within the Task 3 FreeRTOS task context, never from an ISR, avoiding blocking I/O in interrupt context.

3.5 Results

For running the application the following commands were used for compiling, uploading the project and monitoring the port to observe the periodic reports.

```
# to find to which port number the board is connected
arduino-cli board list
# compilation of the program by indicating the target hardware
arduino-cli compile --fqbn esp32:esp32:esp32 sketch
# uploading the program executables to the board
arduino-cli upload -p /dev/cu.usbserial-portnumber --fqbn esp32:esp32:esp32 sketch
# monitoring the port at 115200 baud (ESP32 default)
arduino-cli monitor -p /dev/cu.usbserial-portnumber --config baudrate=115200
```

Below is the output of the commands listed above

```
$ arduino-cli compile --fqbn esp32:esp32:esp32 sketch
Sketch uses 267498 bytes (20%) of program storage space.
Global variables use 21516 bytes (6%) of dynamic memory,
leaving 306164 bytes for local variables.

$ arduino-cli upload -p /dev/cu.usbserial-0001 --fqbn esp32:esp32:esp32 sketch
New upload port: /dev/cu.usbserial-0001 (serial)

$ arduino-cli monitor -p /dev/cu.usbserial-0001 --config baudrate=115200
Using generic monitor configuration.

Monitor port settings:
  baudrate=115200

Connecting to /dev/cu.usbserial-0001. Press CTRL-C to exit.

Lab 4 - FreeRTOS Button Press Monitor
Reports every 10 seconds
--- Report ---
Total presses: 5
Short (<500ms): 3
Long  (>=500ms): 2
Avg duration: 487 ms
```

```
--------------
--- Report ---
Total presses: 0
Short (<500ms): 0
Long  (>=500ms): 0
Avg duration: N/A
--------------
```

The system behavior is as follows:

- When a short press (< 500 ms) is detected, the **green LED** turns on and the red LED turns off. Task 1 gives the binary semaphore and Task 2 picks it up on its next 50 ms tick. The **yellow LED** blinks 5 times rapidly.
- When a long press (≥ 500 ms) is detected, the **red LED** turns on and the green LED turns off. The **yellow LED** blinks 10 times rapidly.
- Every 10 seconds, Task 3 atomically snapshots and resets statistics under the mutex, then prints the report to the serial monitor showing the total number of presses, the count of short and long presses, and the average press duration.
- When no presses occur during a reporting interval, the report shows zeros and "N/A" for the average duration.
- The monitor must be opened at **115200 baud** to receive readable output, as the ESP32 serial interface operates at this rate.

Below there is the visual representation of the circuit
Figure 4 shows the circuit in its idle state with all LEDs off.
A video demonstrating how the program works is posted on youtube hosting platform. Video can be accessed using this link https://youtu.be/teMOPxAjXb4

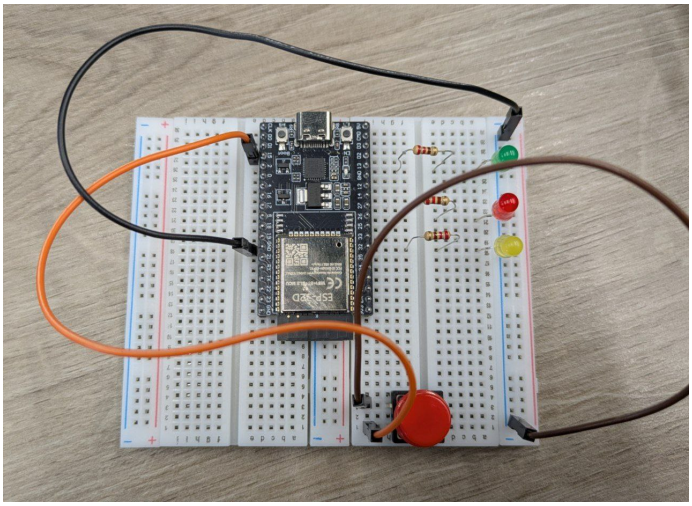

Figure 4: Circuit in idle state

4 Conclusion

Completion of this laboratory work has provided insightful understanding of preemptive multitasking with FreeRTOS on the ESP32 microcontroller and the design principles behind real-time task synchronization.

This fourth laboratory work covered the port of the bare-metal cooperative application to FreeRTOS preemptive tasks, extending it with synchronization mechanisms while preserving the same three-layer architecture and task function structure. The key takeaways are:

1. Porting a **cooperative bare-metal application to FreeRTOS** with minimal changes to task logic – by encapsulating periodicity and offset entirely inside the scheduler service abstraction (`periodicTaskRunner` with `vTaskDelayUntil`), the task functions themselves remain identical in structure to the bare-metal version.

2. Using `vTaskDelayUntil` for drift-corrected periodic execution, which guarantees that each task fires at precise intervals regardless of how long the task body takes to execute – a significant improvement over relative `vTaskDelay`.

3. Implementing **inter-task synchronization** with a binary semaphore for event-driven signaling (Task 1 notifies Task 2 on press detection) and a mutex for protecting shared statistical variables against concurrent access by Tasks 1, 2, and 3.

4. Understanding the difference between a **binary semaphore** (used for signaling, no ownership) and a **mutex** (used for mutual exclusion, with ownership semantics and priority inheritance support in FreeRTOS).

5. Structuring the application into **three distinct layers**: driver, service (scheduler + sync), and application – maintaining the portability and modularity established in Laboratory Work 3.

6. Gaining practical experience with **safe shared resource access** in a preemptive environment, where failing to protect global variables would lead to race conditions and corrupted statistics.

This laboratory work builds directly on Laboratory Work 3, demonstrating how the same application logic and architecture can be lifted from bare-metal to a full RTOS with targeted, minimal changes – a skill essential for developing scalable and maintainable embedded firmware.

References

[1] Espressif Systems. (2023). *ESP32 Series Datasheet*. Retrieved from https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[2] Real Time Engineers Ltd. (2024). *FreeRTOS API Reference*. Retrieved from https://www.freertos.org/a00106.html

[3] Real Time Engineers Ltd. (2024). *FreeRTOS – Using Semaphores and Mutexes*. Retrieved from https://www.freertos.org/Embedded-RTOS-Binary-Semaphores.html

[4] Espressif Systems. (2024). *Arduino core for ESP32*. Retrieved from https://docs.espressif.com/projects/arduino-esp32/en/latest/

[5] Margolis, M. (2011). *Arduino Cookbook: Recipes to Begin, Expand, and Enhance Your Projects*. O'Reilly Media, Inc.

[6] Barr, M. (2006). *Programming Embedded Systems: With C and GNU Development Tools*. O'Reilly Media, Inc.

[7] Barry, R. (2016). *Mastering the FreeRTOS Real Time Kernel – A Hands-On Tutorial Guide*. Real Time Engineers Ltd. Retrieved from https://www.freertos.org/Documentation/RTOS_book.html

A Source Code

A.1 Main Sketch File

```
#include "src/lab4.h"

void setup() {
  lab4Setup();
}

void loop() {
  lab4Loop();
}
```

Listing 1: sketch.ino - Main Arduino Sketch

A.2 Application Module

```
#ifndef LAB4_H
#define LAB4_H

#define SHORT_PRESS_THRESHOLD 500

#define TASK1_REC 10
#define TASK1_OFF 1

#define TASK2_REC 50
#define TASK2_OFF 5

#define TASK3_REC 10000
#define TASK3_OFF 10000

#define TASK1_STACK 2048
#define TASK2_STACK 2048
#define TASK3_STACK 4096

#define TASK1_PRIORITY 3
#define TASK2_PRIORITY 2
#define TASK3_PRIORITY 1

void lab4Setup();
void lab4Loop();

#endif
```

Listing 2: lab4.h - Application Header

```
#include "lab4.h"
#include "drivers/button/button.h"
#include "drivers/led/led.h"
#include "services/scheduler/scheduler.h"
#include "services/serial/serial.h"
#include "services/sync/sync.h"

#include <stdio.h>

static unsigned long lastPressDuration = 0;
static bool lastPressWasShort = false;

static int totalPresses = 0;
static int shortPresses = 0;
static int longPresses = 0;
static unsigned long shortDurationSum = 0;
static unsigned long longDurationSum = 0;

static bool buttonPrevState = false;
static unsigned long pressStartMs = 0;

void task1ButtonDetect() {
  bool pressed = buttonIsPressed();

  if (pressed && !buttonPrevState) {
    pressStartMs = millis();
  } else if (!pressed && buttonPrevState) {
    unsigned long duration = millis() - pressStartMs;
    bool isShort = (duration < SHORT_PRESS_THRESHOLD);

    syncMutexTake();
    lastPressDuration = duration;
    lastPressWasShort = isShort;
    syncMutexGive();

    if (isShort) {
      ledGreenOn();
      ledRedOff();
    } else {
      ledRedOn();
      ledGreenOff();
    }

```

```
44      syncSemaphoreGive();
45    }
46
47    buttonPrevState = pressed;
48  }
49
50  static int yellowBlinksRemaining = 0;
51  static bool yellowIsOn = false;
52
53  void task2Statistics() {
54    if (syncSemaphoreTake(0)) {
55      syncMutexTake();
56      unsigned long duration = lastPressDuration;
57      bool isShort = lastPressWasShort;
58      syncMutexGive();
59
60      syncMutexTake();
61      totalPresses++;
62      if (isShort) {
63        shortPresses++;
64        shortDurationSum += duration;
65        yellowBlinksRemaining = 5 * 2;
66      } else {
67        longPresses++;
68        longDurationSum += duration;
69        yellowBlinksRemaining = 10 * 2;
70      }
71      syncMutexGive();
72
73      yellowIsOn = false;
74    }
75
76    if (yellowBlinksRemaining > 0) {
77      yellowBlinksRemaining--;
78      yellowIsOn = !yellowIsOn;
79      if (yellowIsOn) {
80        ledYellowOn();
81      } else {
82        ledYellowOff();
83      }
84    } else if (yellowIsOn) {
85      ledYellowOff();
86      yellowIsOn = false;
87    }
88  }
89
```

```
90 void task3Report() {
91   syncMutexTake();
92   int rptTotal = totalPresses;
93   int rptShort = shortPresses;
94   int rptLong = longPresses;
95   unsigned long rptShortSum = shortDurationSum;
96   unsigned long rptLongSum = longDurationSum;
97
98   totalPresses = 0;
99   shortPresses = 0;
100  longPresses = 0;
101  shortDurationSum = 0;
102  longDurationSum = 0;
103  syncMutexGive();
104
105  printf("--- Report ---\n");
106  printf("Total presses: %d\n", rptTotal);
107  printf("Short (<500ms): %d\n", rptShort);
108  printf("Long  (>=500ms): %d\n", rptLong);
109
110  if (rptTotal > 0) {
111    unsigned long avgDur = (rptShortSum + rptLongSum) / rptTotal;
112    printf("Avg duration: %lu ms\n", avgDur);
113  } else {
114    printf("Avg duration: N/A\n");
115  }
116
117  printf("--------------\n");
118 }
119
120 void lab4Setup() {
121  setupGreenLed();
122  setupRedLed();
123  setupYellowLed();
124  buttonSetup();
125  srvSerialSetup();
126  syncSetup();
127
128  printf("Lab 4 - FreeRTOS Button Press Monitor\n");
129  printf("Reports every 10 seconds\n");
130
131  TaskDef tasks[] = {
132      {"Task1_Button", task1ButtonDetect, TASK1_REC, TASK1_OFF,
     TASK1_STACK,
133       TASK1_PRIORITY},
134      {"Task2_Stats", task2Statistics, TASK2_REC, TASK2_OFF,
```

```
    TASK2_STACK,
135         TASK2_PRIORITY},
136        {"Task3_Report", task3Report, TASK3_REC, TASK3_OFF,
    TASK3_STACK,
137         TASK3_PRIORITY},
138   };
139
140   srvSchedulerSetup(tasks, 3);
141 }
142
143 void lab4Loop() { vTaskDelay(pdMS_TO_TICKS(1000)); }
```

Listing 3: lab4.cpp - Application Implementation

A.3 Scheduler Service

```
1  #ifndef SCHEDULER_SERVICE_H
2  #define SCHEDULER_SERVICE_H
3
4  #include <freertos/FreeRTOS.h>
5  #include <freertos/task.h>
6
7  typedef void (*TaskFunc)(void);
8
9  typedef struct {
10   const char *name;
11   TaskFunc func;
12   uint32_t periodMs;
13   uint32_t offsetMs;
14   uint32_t stackSize;
15   UBaseType_t priority;
16 } TaskDef;
17
18 void srvSchedulerSetup(TaskDef *tasks, int numTasks);
19
20 #endif
```

Listing 4: scheduler.h - Scheduler Header

```
1  #include "scheduler.h"
2  #include <stdio.h>
3
4  typedef struct {
5    TaskFunc func;
6    uint32_t periodMs;
```

```
  uint32_t offsetMs;
} TaskWrapper;

static void periodicTaskRunner(void *pvParameters) {
  TaskWrapper *ctx = (TaskWrapper *)pvParameters;

  vTaskDelay(pdMS_TO_TICKS(ctx->offsetMs));

  TickType_t xLastWakeTime = xTaskGetTickCount();
  for (;;) {
    ctx->func();
    vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(ctx->periodMs));
  }
}

#define MAX_TASKS 8
static TaskWrapper wrappers[MAX_TASKS];

void srvSchedulerSetup(TaskDef *tasks, int numTasks) {
  for (int i = 0; i < numTasks && i < MAX_TASKS; i++) {
    wrappers[i].func = tasks[i].func;
    wrappers[i].periodMs = tasks[i].periodMs;
    wrappers[i].offsetMs = tasks[i].offsetMs;

    BaseType_t result =
      xTaskCreate(periodicTaskRunner, tasks[i].name, tasks[i].
   stackSize,
                  &wrappers[i], tasks[i].priority, NULL);

    if (result != pdPASS) {
      printf("Failed to create task: %s\n", tasks[i].name);
    }
  }
}
```

Listing 5: scheduler.cpp - Scheduler Implementation

A.4 Sync Service

```
#ifndef SYNC_SERVICE_H
#define SYNC_SERVICE_H

#include <stdbool.h>
#include <stdint.h>

```

```
void syncSetup();

void syncSemaphoreGive();
bool syncSemaphoreTake(uint32_t timeoutMs);

void syncMutexTake();
void syncMutexGive();

#endif
```

Listing 6: sync.h - Sync Service Header

```
#include "sync.h"
#include <freertos/FreeRTOS.h>
#include <freertos/semphr.h>

static SemaphoreHandle_t pressSemaphore = NULL;
static SemaphoreHandle_t statsMutex = NULL;

void syncSetup() {
  pressSemaphore = xSemaphoreCreateBinary();
  statsMutex = xSemaphoreCreateMutex();
}

void syncSemaphoreGive() {
  xSemaphoreGive(pressSemaphore);
}

bool syncSemaphoreTake(uint32_t timeoutMs) {
  return xSemaphoreTake(pressSemaphore, pdMS_TO_TICKS(timeoutMs)) ==
    pdTRUE;
}

void syncMutexTake() {
  xSemaphoreTake(statsMutex, portMAX_DELAY);
}

void syncMutexGive() {
  xSemaphoreGive(statsMutex);
}
```

Listing 7: sync.cpp - Sync Service Implementation

A.5 Button Driver

```h
#ifndef BUTTON_DRIVER_H
#define BUTTON_DRIVER_H

#define BUTTON_PIN 15

void buttonSetup();
bool buttonIsPressed();

#endif
```

Listing 8: button.h - Button Driver Header

```cpp
#include "button.h"
#include <Arduino.h>

void buttonSetup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

bool buttonIsPressed() {
  return digitalRead(BUTTON_PIN) == LOW;
}
```

Listing 9: button.cpp - Button Driver Implementation

A.6 LED Driver

```h
#ifndef LED_DRIVER_H
#define LED_DRIVER_H

#include <Arduino.h>

#define LED_GREEN_PIN 13
#define LED_RED_PIN 26
#define LED_YELLOW_PIN 32

void setupGreenLed();
void setupRedLed();
void setupYellowLed();

void ledGreenOn();
void ledGreenOff();
bool ledGreenIsOn();

```

```
18 void ledRedOn();
19 void ledRedOff();
20
21 void ledYellowOn();
22 void ledYellowOff();
23
24 #endif
```

Listing 10: led.h - LED Driver Header

```
1  #include "led.h"
2  #include <Arduino.h>
3
4  int ledPinRed;
5  int ledPinGreen;
6  int ledPinYellow;
7
8  void setupGreenLed() {
9    ledPinGreen = LED_GREEN_PIN;
10   pinMode(ledPinGreen, OUTPUT);
11 }
12
13 void setupRedLed() {
14   ledPinRed = LED_RED_PIN;
15   pinMode(ledPinRed, OUTPUT);
16 }
17
18 void setupYellowLed() {
19   ledPinYellow = LED_YELLOW_PIN;
20   pinMode(ledPinYellow, OUTPUT);
21 }
22
23 void ledGreenOn() {
24   digitalWrite(ledPinGreen, HIGH);
25 }
26 void ledGreenOff() {
27   digitalWrite(ledPinGreen, LOW);
28 }
29
30 bool ledGreenIsOn() {
31   return digitalRead(ledPinGreen) == HIGH;
32 }
33
34 void ledRedOn() {
35   digitalWrite(ledPinRed, HIGH);
36 }
```

```
37 void ledRedOff() {
38   digitalWrite(ledPinRed, LOW);
39 }
40
41 void ledYellowOn() {
42   digitalWrite(ledPinYellow, HIGH);
43 }
44 void ledYellowOff() {
45   digitalWrite(ledPinYellow, LOW);
46 }
```

Listing 11: led.cpp - LED Driver Implementation

A.7 Serial Service

```
1 #ifndef SERIAL_SERVICE_H
2 #define SERIAL_SERVICE_H
3
4 void srvSerialSetup();
5
6 #endif
```

Listing 12: serial.h - Serial Service Header

```
1 #include "serial.h"
2 #include <Arduino.h>
3
4 void srvSerialSetup() {
5   Serial.begin(115200);
6 }
```

Listing 13: serial.cpp - Serial Service Implementation